CS-202: PROGRAMMING PARADIGM AND PRAGMATICS

Static Analyzer & Optimizer

Converting Raw C Code to an Optimized Control Flow Graph

Group - 16 Members

Harshit Sahu (2024csb1203)

Sarthak Keshaowar (2024csb1215)

Aryan Goyal (2024csb1102)

Agampreet Singh (2024csb1097)

Abhishulesh Gevin (2024csb1093)

1. Project Overview

The **Static Analyzer & Optimizer** is a comprehensive compiler toolchain project engineered to parse, analyze, and optimize raw C source code. Rather than constructing a C parser from scratch, the architecture is built upon the robust, industry-standard **LLVM/Clang infrastructure**, enabling highly accurate Abstract Syntax Tree (AST) generation and traversal.

The tool processes raw code by translating it into a Control Flow Graph (CFG) comprised of distinct basic blocks. Through rigorous static dataflow analysis (specifically Live Variable Analysis), the engine determines the lifecycle and reachability of all variables. Leveraging this data, aggressive optimization techniques—including Constant Folding, Constant Propagation, and Dead Code Elimination—are applied to physically refactor the code. Finally, a Python-based interactive web dashboard (built with Streamlit) bridges the backend C++ engine with a user-friendly frontend, providing real-time, side-by-side visualizations of the internal graph and the optimized source code.

2. Project Goals

The primary objectives of this project were structured across three distinct phases of compiler design:

- **Phase 1 (The Foundation):** Successfully interface with the Clang frontend to parse a C codebase, isolate basic blocks, and construct an accurate intra-procedural Control Flow Graph.

- **Phase 2 (Static Analysis):** Implement backward dataflow equations to compute Block-Level *Gen* and *Kill* sets, establishing a reliable Live Variable Analysis framework to track variable states across execution paths.
- **Phase 3 (Optimizations):** Utilize the `clang::Rewriter` class to physically transform the AST, substituting constants and eliminating mathematically redundant or structurally unreachable code.
- **Visualization & Tooling:** Develop an intuitive Graphical User Interface (GUI) to visualize the internal CFG using Graphviz (DOT format) and seamlessly present the optimized output code to the user.

3. File Structure

The repository strictly separates the C++ LLVM backend logic from the Python web frontend and build configurations.

```
StaticAnalyzerProject/
    ├── CFGBuilder.cpp    // Core C++ backend traversing AST and applying CFG analysis.
    ├── CMakeLists.txt    // Build configuration linking LLVM/Clang libraries.
    ├── webinterface.py   // Streamlit dashboard providing the UI and handling subprocesses.
    └── optimized.c       // Output file generated dynamically by the Rewriter.
```

4. Implementation & System Architecture

The core of the project relies on a pipeline that bridges a modern Python-based web interface with a high-performance C++ static analysis engine. This section details the architectural choices, data structures, and algorithmic passes used to achieve the code transformations.

4.1. The Pipeline Architecture

The system operates on a dual-language architecture to separate computational heavy-lifting from user interaction:

- **Frontend (Python/Streamlit):** Manages state and user inputs. It accepts raw C code, writes it to a temporary file (`temp.c`), and invokes the compiled C++ binary as a subprocess. It then parses standard output (stdout) for graph generation and reads the modified files.
- **Backend (C++/LLVM):** The `CFGBuilder.cpp` executable acts as a Clang Tool. It hooks into the compiler instance using a custom `ASTFrontendAction` and `ASTConsumer`. This ensures that our code has full access to the precise, typed AST generated by Clang's lexer and parser.

4.2. Phase 1: The Foundation (AST Traversal and CFG Construction)

Parsing raw C text into a manageable graph is the foundational step:

1. **Visitor Pattern:** The tool defines a `RecursiveASTVisitor` named `MyASTVisitor`. As the AST is traversed, it targets `FunctionDecl` nodes to isolate individual functions (e.g., `main`, user-defined functions).
2. **CFG Instantiation:** For each function body, the tool utilizes Clang's built-in `clang::Analysis::CFG::buildCFG` method. This automatically segments sequential statements into Basic Blocks (`CFGBlock`), inherently resolving branches, loops, and conditional jumps into directional edges.
3. **Reachability Analysis:** A Breadth-First Search (BFS) starting from the CFG's Entry block maps all reachable blocks, storing them in a `visitedBlocks` set. Any block not in this set is intrinsically dead code (unreachable).

4.3. Phase 2: Static Analysis (The 4-Pass Dataflow Engine)

The centerpiece of the architecture is the multi-pass dataflow analysis engine. It must determine not just what variables exist, but their lifespan and runtime values to inform safe optimizations.

Pass 1: Forward Pass (Constant Folding & Propagation)

The engine iterates forward through the reachable CFG blocks. It implements a custom evaluation helper (`evaluateExpr`) that dynamically resolves `IntegerLiteral`, `DeclRefExpr`, and `BinaryOperator` nodes.

- If an expression (e.g., `int x = 3 + 5;`) can be evaluated to a constant, the value is cached in a `currentConsts` map, and the `clang::Rewriter` instantly modifies the source text to `8`.
- If a variable's value is altered dynamically (e.g., `x = y + 2;` where `y` is unknown), the variable is purged from the constants map to prevent invalid propagation.

Pass 2: Backward Pass (Gen/Kill Set Construction)

To track variable lifespans, the engine traverses the CFG blocks backward. For every block, it generates:

- **Def Sets (Kill):** Variables assigned within the block (e.g., LHS of `BinaryOperator`, or `VarDecl` initializations).
- **Use Sets (Gen):** Variables read within the block (extracted via the `extractActualUses` recursive helper).

Crucially, the logic strictly respects intra-block statement ordering. A definition of a variable inside a block "kills" prior uses within that same block when calculating the overall block-level sets.

Pass 3: Fixed-Point Iteration (Liveness Calculation)

With Block-Level Gen/Kill sets established, the engine applies standard dataflow equations iteratively until convergence (no further changes occur to any sets):

```
Out[B] = ∪ In[S] for all successors S of B
In[B] = Use[B] ∪ (Out[B] - Def[B])
```

This globally resolves exactly which variables hold values that will be read in the future paths of the program.

4.4. Phase 3: Optimizations

Pass 4: Backward Pass (Statement-Level Dead Code Elimination)

The engine revisits each statement backward. If a statement defines a variable (e.g., `y = 10;`), but that variable is *not* present in the computed `OutLive` set for that point in execution, the statement is deemed useless. The `Rewriter` targets the `SourceRange` of the AST node and overwrites it with the comment `/* [DEAD CODE REMOVED] */`.

Global Optimizations: Unreachable Function Pruning

Beyond statement-level DCE, the architecture tracks function calls globally. The `VisitCallExpr` hook populates a `calledFunctions` set. In the final phase of the translation unit analysis, any defined function not in this set (excluding the `main` entrypoint) has its entire source range replaced with a removal comment, dramatically reducing binary bloat.

Optimization Technique	Implementation Summary
Constant Folding	Evaluates deterministic arithmetic expressions at compile time using dynamic AST tracking (Pass 1).
Constant Propagation	Physically substitutes downstream reads of a variable with its definitively known literal value (Pass 1).
Dead Code Elimination	Removes assignments to variables that are never subsequently read before exiting or being reassigned (Pass 4).
Unreachable Code / Functions	Prunes CFG nodes lacking paths from the Entry block, and removes unused user-defined functions globally.

5. Graphical Visualization Engine (Web Dashboard)

To fulfill the visualization requirement, the backend generates raw DOT syntax printed directly to `stdout`. The frontend Streamlit application captures this output and utilizes the `graphviz` library to dynamically render the DOT CFG in the browser.

- **Node Shapes:** Standard statements are boxes; Entry/Exit blocks are ellipses; Conditional branch blocks (terminators) are diamonds. I/O functions (like `printf` or `scanf`) are identified via call graph analysis and styled as parallelograms.
- **Color Coding:** Dead/Unreachable blocks are marked with a distinct red background.
- **Live Variable Annotations:** Every generated node is appended with its respective `Live IN` and `Live OUT` variable sets computed from Pass 3, acting as an invaluable debugging and educational tool.

6. Bonus Features & Expansions

> **DOMINATOR TREE LOOP DETECTION**
>
> The CFG builder computes Dominator sets for all blocks (a block *D* dominates *B* if every path to *B* must pass through *D*). By intersecting dominator sets iteratively, the tool identifies back-edges (where a block points to its dominator). These are explicitly rendered in the DOT graph with thick blue lines, proving the existence of loops (like `while` and `for` constructs).

Security Taint Analysis Integration: Alongside the core optimizations, foundational work was implemented to trace user input. By tracking inputs (such as `scanf` nodes) and verifying their reachability to execution sinks, the tool lays the groundwork for basic security taint analysis to identify unsanitized data flows.

7. How to Compile and Run

The project uses `CMake` to manage LLVM and Clang dependencies. Follow these steps to build the backend analyzer and launch the interactive dashboard.

Step 1: Build the C++ Backend

Ensure that LLVM and Clang (version 10 or higher) are installed on your system. Run the following commands in the project root directory:

```
# Create a build directory and navigate into it
$ mkdir build && cd build

# Generate Makefiles using CMake
$ cmake ..

# Compile the analyzer executable
$ make
```

Step 2: Launch the Web Dashboard

Ensure that Python and `streamlit` are installed (`pip install streamlit graphviz`). Ensure that the generated `analyzer` executable is placed in the same directory as the Python script.

```
# Run the Streamlit interface
$ streamlit run webinterface.py
```

Step 3: Usage

Once the local server launches in your browser, paste any raw C code snippet into the left-hand editor pane and click **"Analyze & Generate CFG"**. The tool will parse the code, generate the visual graph on the first tab, and display the transformed, optimized C source code on the second tab.

8. Conclusion

The developed system successfully satisfies all critical requirements of the project. By combining the rigorous parsing capabilities of LLVM/Clang with standard dataflow analysis paradigms, the tool accurately generates Control Flow Graphs and successfully performs non-trivial source-to-source code optimizations. The addition of dominator-based loop detection, security analysis tracking, and a responsive web dashboard makes the tool both an effective educational visualizer and a highly functional static analyzer prototype.